

Venus Executor Audit Report

Please Note

1. The analysis of the Severity is purely based on the smart contracts mentioned in the Audit Scope and does not include any other potential contracts deployed by the Owner. No applications or operations were reviewed for Severity. No product code has been reviewed.
2. Due to the time limit, the audit team did not do much in-depth research on the business logic of the project. It is more about discovering issues in the smart contracts themselves.

Audit Period: 2026/05/05 - 2026/05/08 (YYYY/MM/DD)

Overall Risk: **Informational**

Project Name	Venus Executor Audit Report
Github	https://github.com/VenusProtocol/venus-periphery/pull/61

Smart contracts list:

No.	Contract Name	Verdict	Details
1	Executor.sol	Informational	[I01] [I02] [I03] [I04] [I05] [I06] [I07]
2	IExecutor.sol	Green	

Findings

[I01] Misleading LTVAdjusted event in handleLTVAdjust when no pool is tightened

Contract Executor.sol

Severity Level Informational

Description

Description

The Executor.handleLTVAdjust() function routes through EBrake.decreaseCF, which enforces tighten only LTV adjustments on both branches.

- On the IL branch, decreaseCF reverts when the new CF exceeds the current CF and is a no-op when they are equal.
- On the Core markets branch, decreaseCF operates per pool and silently skips any pool where the new CF is $\geq$ the current CF.

JavaScript

```
/// @inheritdoc IExecutor
function handleLTVAdjust(address market, uint256 adjustedLTV)
external nonReentrant {
    _checkAccessAllowed("handleLTVAdjust(address,uint256)");

    _checkAndGetConfig(market);

    // EBrake.decreaseCF iterates all pools (including e-mode) and
    no-ops per-pool
    // when already at or below adjustedLTV - idempotency is
    guaranteed on EBrake's side.
    EBrake.decreaseCF(market, adjustedLTV);
    emit LTVAdjusted(msg.sender, market, adjustedLTV);
}
```

As a result, when adjustedLTV is $\geq$ the current CF for every pool on the core markets, no state change occurs, yet handleLTVAdjust still emits the LTVAdjusted event.

Off-chain consumers observing this event will be misled into believing a mutation took place when none did. This is inconsistent with handleCapAdjust, which returns early without emitting in the equal cap case.

Recommendation Propagate a boolean return value from EBrake.decreaseCF indicating whether any pool was actually tightened, and have handleLTVAdjust either skip the event emission or revert when no pool was modified.

Status Acknowledged, no fix implemented. In this case, the permissioned caller of handleLTVAdjust can ensure accurate inputs to prevent unwanted event emissions

[I02] setMarketConfig only validates market listing against the core pool

Contract Executor.sol

Severity Level Informational

Description

Description

The Executor.setMarketConfig() function validates that a market is listed on Diamond by querying poolMarkets(corePoolId, market), which only checks the core pool. A market that is listed exclusively in an e-mode pool with poolId > 0 and is not present in the core pool will be rejected with MarketNotListed.

While current Venus operations ensure that every e-mode market is also registered in the core pool, if a future deployment introduces an e-mode-only market, the Executor would be unable to register a configuration for it, blocking legitimate operations.

The only edge case where this can occur currently is when an admin configuration occurs where MarketFacet.unlistMarket sets isListed = false on the core-pool entry and does not cascade through e-mode pools. So an admin sequence of _addPoolMarket(emodelId, vToken) followed by unlistMarket(vToken) would leave the market listed only in the e-mode pool, and the Executor would then reject any subsequent setMarketConfig call for it.

JavaScript

```
function setMarketConfig(address market, MarketConfig calldata config)
external {

    _checkAccessAllowed("setMarketConfig(address,(uint256,uint256,bool))");
    if (market == address(0)) revert ZeroAddress();

    if (IS_CORE_POOL) {
        (bool isListed, , , , , ) =
        COMPTROLLER.poolMarkets(COMPTROLLER.corePoolId(), market);
        if (!isListed) revert MarketNotListed(market);
    } else {
        IILComptroller.Market memory m =
        IILComptroller(address(COMPTROLLER)).markets(market);
        if (!m.isListed) revert MarketNotListed(market);
    }

    _isConfigured[market] = true;
    marketConfigs[market] = config;
    emit MarketConfigSet(market, config);
}
```

Recommendation Consider documenting the assumption that every e-mode market must also be a core pool market explicitly in the contract, or loosen the validation in setMarketConfig() to

scan the range [corePoolId, lastPoolId] and accept the market as valid if any pool reports an isListed entry for it.

Status Acknowledged, no fix implemented, given the current e-mode pool addition path in Venus enforces core-pool addition first, and Venus do not plan to have a e-mode pool not listed within the core pool as well

[I03] Cap exceeding halt handlers in Executor can be evaded by sandwiching the transaction

Contract Executor.sol

Severity Level Informational

Description

Description

The handleSupplyCapExceeding() and handleBorrowCapExceeding() functions gate execution on a strict on chain check that the cap is currently breached.

- The supply variant computes $\text{totalSupply} * \text{exchangeRateCurrent()} / 1e18$ and reverts with CapNotBreached when this value is below supplyCap.
- The borrow variant reverts when totalBorrowsCurrent is below borrowCap.

Because both are evaluated against current chain state at execution time, any market participant who observes the handling transactions adjusting supply and borrow cap in the public mempool can front run it with a redeem or repayBorrow call sized to drop the metric one wei below the cap, forcing the handler to revert.

The attacker can then back run with mint or borrow to restore their position up to the cap. The pause action set on EBrake cannot close this window, since pauseSupply only pauses MINT and pauseBorrow only pauses BORROW.

REDEEM and REPAY remain open by design, which means no configuration of pause flags on EBrake prevents the front run. The impact is limited to DoS rather than direct value loss, but it extends the survival time of any underlying issue the handling was meant to contain during emergencies

JavaScript

```
/// @inheritdoc IExecutor
function handleSupplyCapExceeding(address market) external
nonReentrant {
    _checkAccessAllowed("handleSupplyCapExceeding(address)");

    _checkAndGetConfig(market);

    IComptroller comptroller = IComptroller(address(COMPTROLLER));
    // exchangeRateCurrent() accrues interest as a side effect before
    the cap comparison.
```

```

        uint256 supplyUnderlying = (IVToken(market).totalSupply() *
IVToken(market).exchangeRateCurrent()) / 1e18;
        uint256 supplyCap = comptroller.supplyCaps(market);

        // supplyCap == 0 is treated as a misconfiguration (no Venus
market is intentionally
        // uncapped in production); halt is permitted so the market can
always be frozen.
        // Accidental halt of a genuinely-uncapped market is recoverable
via governance VIP,
        // whereas leaving a draining market un-halttable is not.
        if (supplyCap != 0 && supplyUnderlying < supplyCap) revert
CapNotBreached();

        EBRAKE.pauseSupply(market);
        EBRAKE.decreaseCF(market, 0);
        emit SupplyCapExceeding(msg.sender, market);
    }

    /// @inheritdoc IExecutor
    function handleBorrowCapExceeding(address market) external
nonReentrant {
        _checkAccessAllowed("handleBorrowCapExceeding(address)");

        _checkAndGetConfig(market);

        IComptroller comptroller = IComptroller(address(COMPTROLLER));
        uint256 totalBorrows = IVToken(market).totalBorrowsCurrent();
        uint256 borrowCap = comptroller.borrowCaps(market);

        // borrowCap == 0 is treated as a misconfiguration - see
handleSupplyCapExceeding.
        if (borrowCap != 0 && totalBorrows < borrowCap) revert
CapNotBreached();

        EBRAKE.pauseBorrow(market);

        emit BorrowCapExceeding(msg.sender, market);
    }

```

Recommendation	Likely no fix required, given the issue can be prevented by admin pausing redemption/repay via the SetterFacet <code>_setActionsPaused</code> functions or <code>Ebrake.pauseRedeem</code> for redemptions. If not, a private transaction can be utilized when performing CF disablement and supply/borrow pauses when caps are breached.
Status	Acknowledged, no fix implemented, Venus has indicated that the keeper will submit

required transactions via private RPC, and the governance also retains the ability to call pauseSupply/pauseBorrow/decreaseCF directly via a VIP.

[I04] Missing zero value check on minBorrowCap and minSupplyCap in setMarketConfig

Contract Executor.sol

Severity Level Informational

Description

Description

The Executor.setMarketConfig() function does not enforce a non zero floor on MarketConfig.minBorrowCap or MarketConfig.minSupplyCap.

Both handleSupplyCapExceeding() and handleBorrowCapExceeding() rely on an on chain breach check that compares the current supply or borrow against the cap and reverts with CapNotBreached when no breach exists. These breach handlers also intentionally treat a cap value of 0 as a misconfiguration and permit a halt in that case, since no Venus market is meant to be uncapped in production.

If minSupplyCap == 0 is allowed through setMarketConfig(), a keeper can call handleCapAdjust(market, SUPPLY, 0). The bound check adjustedCap < config.minSupplyCap does not trigger, because 0 < 0 is false, so the call routes through EBRAKE.setMarketSupplyCaps([market], [0]) and the on chain supply cap becomes 0. After that, any caller authorised for handleSupplyCapExceeding() can halt the market, because the cap is now 0 and the breach check is bypassed. The same path exists symmetrically for minBorrowCap == 0 and handleBorrowCapExceeding().

The impact is that a zero floor in the market configuration gives keepers a two step path to halt a market without ever proving an actual cap breach, bypassing the only on chain integrity check that the breach handlers depend on.

JavaScript

```
function setMarketConfig(address market, MarketConfig calldata config)
external {

    _checkAccessAllowed("setMarketConfig(address,(uint256,uint256,bool))");
    if (market == address(0)) revert ZeroAddress();

    if (IS_CORE_POOL) {
        (bool isListed, , , , , ) =
        COMPTROLLER.poolMarkets(COMPTROLLER.corePoolId(), market);
        if (!isListed) revert MarketNotListed(market);
    } else {
        IILComptroller.Market memory m =
        IILComptroller(address(COMPTROLLER)).markets(market);
        if (!m.isListed) revert MarketNotListed(market);
    }
}
```

```

        _isConfigured[market] = true;
        marketConfigs[market] = config;
        emit MarketConfigSet(market, config);
    }

```

Recommendation Consider rejecting any configuration where `minBorrowCap == 0` or `minSupplyCap == 0` by reverting with a dedicated error within `setMarketConfig()`. If not, extend the documentation to also include a `minBorrowCap/minSupplyCap` as a misconfiguration as well.

Status Acknowledged, no fix implemented, Venus has stated that the governance is trusted to not misconfigure `minBorrowCap/minSupplyCap` values.

[I05] Missing constructor consistency checks between Executor and EBrake allow mismatched pairings

Contract Executor.sol

Severity Level Informational

Description

The Executor constructor accepts an IEBrake instance, an ICorePoolComptroller instance, and an `isCorePool_` flag, but never verifies that these three values are mutually consistent with the paired EBrake deployment.

Two invariants are silently relied on.

- First, the comptroller passed to Executor must equal `eBrake_.COMPTROLLER()`, otherwise the Executor validates against one comptroller while routing tightening calls to an EBrake that operates on a different comptroller.
- Second, the pool type flag must agree across both contracts, meaning `eBrake_.IS_ISOLATED_POOL()` must be the negation of `isCorePool_`, otherwise the Executor performs core pool listing checks while EBrake follows the isolated pool ABI path, or vice versa.

Without these checks, a deployment time misconfiguration can produce an Executor and EBrake pair that results in inconsistencies

JavaScript

```

    /// @notice Deploy Executor with the EBrake contract and comptroller
    type.
    /// @param eBrake_ The EBrake contract address.

```

```

    /// @param comptroller_ The comptroller that EBrake operates on.
    /// @param isCorePool_ True if targeting BSC Diamond comptroller,
    false for IL comptroller.
    /// @custom:oz-upgrades-unsafe-allow constructor
    constructor(IEBrake eBrake_, ICorePoolComptroller comptroller_, bool
isCorePool_) {
        if (address(eBrake_) == address(0)) revert ZeroAddress();
        if (address(comptroller_) == address(0)) revert ZeroAddress();

        EBRAKE = eBrake_;
        COMPTROLLER = comptroller_;
        IS_CORE_POOL = isCorePool_;

        _disableInitializers();
    }

```

Recommendation Consider adding two require style checks at the end of the Executor constructor before assigning the immutables.

```

JavaScript
    constructor(IEBrake eBrake_, ICorePoolComptroller comptroller_, bool
isCorePool_) {
        if (address(eBrake_) == address(0)) revert ZeroAddress();
        if (address(comptroller_) == address(0)) revert ZeroAddress();

        EBRAKE = eBrake_;
        if (eBrake_.COMPTROLLER() != comptroller_)
            revert ComptrollerMismatch();
        if (eBrake_.IS_ISOLATED_POOL() == isCorePool_)
            revert PoolTypeMismatch();
        COMPTROLLER = comptroller_;
        IS_CORE_POOL = isCorePool_;

        _disableInitializers();
    }

```

Status Acknowledged, no fix implemented, Venus has stated that they will ensure accurate address inputs during deployment.

[I06] handleBorrowCapExceeding Can Silently Succeed for a Delisted Market

Contract Executor.sol

Severity Level **Informational**

Description	Description setMarketConfig validates that a market is listed in the Comptroller at configuration time, but <code>_isConfigured[market]</code> is never cleared if the market is subsequently delisted via a governance VIP. Among the four handler functions, <code>handleBorrowCapExceeding</code> is the only one that can silently succeed for a delisted market: <code>borrowCaps</code> returns 0 for a delisted market, which the code treats as a misconfiguration and permits execution to continue. EBrake's <code>_pauseSingleAction</code> performs no listing check and calls <code>setActionsPaused</code> directly, allowing the transaction to succeed and emit a <code>BorrowCapExceeding</code> event — with no actual protective effect on the delisted market. The other three handlers behave predictably: <code>handleLTVAdjust</code> and <code>handleSupplyCapExceeding</code> will revert because EBrake's <code>decreaseCF</code> checks listing status internally; <code>handleCapAdjust</code> will be rejected by EBrake with <code>CapExceedsCurrent</code> since <code>borrowCaps</code> returns 0 while <code>adjustedCap > 0</code>. <pre> JavaScript // Executor - handleBorrowCapExceeding _checkAndGetConfig(market); // passes - _isConfigured is still true uint256 totalBorrows = IVToken(market).totalBorrowsCurrent(); uint256 borrowCap = comptroller.borrowCaps(market); // returns 0 for delisted market if (borrowCap != 0 && totalBorrows < borrowCap) revert CapNotBreached(); // borrowCap == 0 → treated as misconfiguration, execution continues EBRAKE.pauseBorrow(market); // _pauseSingleAction has no listing check, may succeed emit BorrowCapExceeding(msg.sender, market); // misleading event emitted </pre>
Recommendation	Include a call to <code>setMarketConfig(market, MarketConfig({enabled: false, ...}))</code> in any governance VIP that delists a market, and document this as a required operational step to keep Executor's configuration in sync with the Comptroller's state.
Status	Acknowledged, no fix implemented, Venus has stated that they will ensure accurate operational actions during a market disablement.

[I07] Missing Executor functionalities for emergency actions

Contract	Executor.sol
Severity Level	Informational
Description	Description In the Venus On-chain monitoring framework documentation, it documents the Hypernative (off-chain signal S1-S8), that is triggered when certain composite alert

[conditions](#) are met to perform specific market emergency responses. The exact actions are documented below

Scenario	Signals Fired	Severity	Action
C1 + C2 + C3	S1, S2, S3, S7	 P0	Swap → borrow exploitation pattern confirmed; pause CF and disable borrowable on affected market
C4a alone	S4 (supply)	 P1	Halt supply + CF → 0; verify if organic or adversarial
C1 + C2	S1, S2, S7	 P1	Pause market
C1 + C4a	S1, S4, S7	 P0	Swap → supply cap exploit confirmed; pause CF and disable borrowable
All conditions	S1–S7	 P0	Pause all affected markets; broadcast incident alert
S8 alone	S8	 P2	Flag wallet; log interaction; notify compliance team; monitor subsequent actions from same wallet
S8 + S1 (large swap from mixer-linked wallet)	S8, S1	 P1	Pre-attack positioning pattern; elevate all thresholds for this wallet's activity; cross-check with S7 for existing positions

S8 + S5 (large borrow from mixer-linked wallet)	S8, S5	 P1	Suspicious large capital deployment from high-risk wallet; flag market; cross-check with C1 for coordinated attack signal
S8 + any Critical condition	S8 + C1/C2/C4a	 P0	Mixer-funded attacker confirmed active; trigger e-brake immediately; broadcast incident alert

Several Actions are Covered by the Executor contract, namely:

- C2 (LTV tightening per $(1/LTV - 1)$) → handleLTVAdjust
- C3 (tighten caps) → handleCapAdjust (bounded by minSupplyCap/minBorrowCap)
- C4a (supply cap breach: halt supply + CF→0) → handleSupplyCapExceeding (exact match)
- C4b (borrow cap breach: pause borrow) → handleBorrowCapExceeding
- S6 ($HF \leq 1.1$ pre-emptive) → translates to handleLTVAdjust or handleCapAdjust

There is a single scenario not covered by the Executor contract, namely focusing on unconditional pauseBorrow / pauseSupply calls.

- handleBorrowCapExceeding only fires EBRAKE.pauseBorrow when $totalBorrows \geq borrowCap$ (or $cap == 0$)
- handleSupplyCapExceeding only fires EBRAKE.pauseSupply when $supplyUnderlying \geq supplyCap$ (or $cap == 0$)

This blocks a single scenario from firing as written:

- C1 + C2 → "Pause market" (no cap breach yet)
- C1 + C2 + C3 → "pause CF and disable borrowable on affected market" (no cap breach yet)

Recommendation Consider adding unconditional handlePauseBorrow(market) and handlePauseSupply(market) handlers that route directly to EBRAKE.pauseBorrow and EBRAKE.pauseSupply without requiring a cap breach, protected by off-chain signals and gated by the existing ACM permissions and _checkAndGetConfig validation.

Additionally, to support multi market pauses, consider batch variants that accept an array of markets in a single call to handle the “All conditions” scenario

Status

Venus has indicated that this is not an issue, as the missing coverage is present within the DeviationSentinel contract and is not expected to be covered by the Executor contract
